

Module 02 — Data Loader

src/data_loader.py • 70 lines

In one sentence

Reads the three CSV files. The smallest module, with one line that prevents a serious bug.

1. What this module does

Every other module needs the data, and none of them should each write their own file-reading code. This module is the single place where the three CSVs are read.

It is only 70 lines, but it does two things that matter. First, it works out the file paths from its *own* location, so the project runs from any folder or drive. Second, it converts the timestamp column into a real date type as the data comes in.

2. Concepts covered

Every idea this module uses, in plain terms.

Concept	In simple terms	Where it is used here
Single responsibility	One job, done in one place	Only this file reads the CSVs
Relative paths	Work out where you are, do not hardcode it	<code>os.path.dirname(os.path.abspath(__file__))</code>
Type parsing at the boundary	Convert data as it enters the system	<code>parse_dates</code> on the timestamp column
Fail fast with a useful message	Say what is wrong and how to fix it	<code>check_file()</code> names the command to run

3. How it works, step by step

1. **Work out the project root** by going up one folder from this file.
2. **Build the three file paths** from that root.
3. **Check the file exists** before reading, and if not, raise an error that names the exact command needed to create it.
4. **Read the CSV** with pandas, converting the timestamp column to a real datetime.

4. Key functions

Function	What it does
<code>check_file()</code>	Raises a helpful error if a file is missing
<code>load_users()</code>	Reads users.csv
<code>load_items()</code>	Reads items.csv
<code>load_interactions()</code>	Reads interactions.csv with real dates
<code>load_all()</code>	Returns all three tables at once

5. Inputs, outputs and how to run

	Detail
Inputs	data/raw/*.csv
Outputs	Three pandas DataFrames
Runtime	Under a second
Run with	python src/data_loader.py

6. Key code

src/data_loader.py — load_interactions()

```
43 | def load_interactions() -> pd.DataFrame:
44 |     """Load the interaction log."""
45 |     check_file(INTERACTIONS_FILE, "interactions.csv")
46 |     # parse_dates matters: as strings the time-based split compares
47 |     # lexicographically and silently produces a wrong cutoff.
48 |     return pd.read_csv(INTERACTIONS_FILE, parse_dates=["timestamp"])
```

The parse_dates argument is the important part of this whole module.

7. Things to remember

The one line that prevents a real bug

Without `parse_dates=["timestamp"]`, pandas reads the timestamps as **text**. The train/test split later compares timestamps with `<=`. Comparing text works alphabetically, which *happens* to give the right answer for dates written as YYYY-MM-DD - so the bug would never raise an error. It would just quietly produce a slightly wrong split. Converting at the point of reading makes the mistake impossible.

8. Likely questions

Q. Why not just call `pd.read_csv` wherever you need the data?

A. Because then the file path, the date parsing and the error handling would be copied into a dozen places, and they would drift apart. One loader means one place to change if the schema or the location moves.

Q. Why compute paths instead of writing them out?

A. The previous version of this project hardcoded an absolute path to a folder that did not exist on this machine, and every single module failed on import. Computing from `__file__` means it runs anywhere.